

FitTrack Analytics

Feature Overview & Personal Trainer Usability Report

27 mars 2026

1. Current Features

FitTrack Analytics is a modern fitness tracking application built with Next.js 14, TypeScript, MongoDB, and Tailwind CSS. The following features are already available in the application.

%¶ Authentication with NextAuth

Secure credentials-based login and user registration powered by NextAuth.js. Passwords are hashed with bcryptjs, and session data is stored server-side.

%¶ Workout Logging

Users can create strength and cardio workout entries. An exercise autocomplete backed by the open wger API helps users quickly find and select exercises.

%¶ Nutrition Logging

Meal logging with automatic calculation of total calories, protein, carbohydrates, and fat. Supports food search for rapid entry.

%¶ Progress Tracking & Dashboards

Weight and body-measurement tracking over time with interactive trend charts (Recharts). Dashboard overview cards summarise recent workouts and progress at a glance.

%¶ Settings & Dark Mode

Profile details, unit preferences (metric/imperial), and dark-mode preference are all persisted in MongoDB and applied across the app.

2. Proposed New Features for Personal Trainer Usability

To make FitTrack Analytics a compelling tool for personal trainers (PTs) and their clients, the following features are recommended for the next development phase.

%¶ Custom Workout Plans for Clients

Allow PTs to build and assign structured, multi-week workout programmes to individual clients. Plans should support drag-and-drop day/week scheduling, exercise selection via the wger library, set/rep targets, and version history so progress against a plan can be measured.

%¶ Client Profile & Progress Summary

A dedicated PT dashboard that lists all connected clients with key metrics (latest weight, workouts completed this week, adherence rate). Drill-down into any client profile to view their full workout, nutrition, and body-measurement history. Generate printable or shareable weekly/monthly progress reports.

%¶ Direct Communication Between Trainers and Clients

An in-app messaging system that lets PTs send feedback after each logged session, answer client questions, and share motivational content. Push notifications (web or email) keep both parties informed without leaving the platform.

%¶ Trainer & Client Roles

Extend the authentication layer to support two distinct user roles: Trainer and Client. PTs can invite clients by email, and each client can belong to one or more trainers. Role-based access control ensures clients only see their own data while trainers see all connected clients.

%¶ Video & Exercise Library

A curated library where PTs can upload instructional video clips or link to external demonstrations. Videos can be attached directly to exercises inside a training plan so clients always have the correct technique reference.

%¶ Gamification & Motivation

Achievement badges, streak counters, and weekly challenge boards to keep clients engaged. PTs can create custom challenges (e.g., '5 sessions this week') with optional leaderboards among a client group.

3. Tech Stack & Functionality Overview

The current architecture provides a solid foundation for all proposed features. Below is a summary of each layer.

%¶ Frontend – Next.js & Tailwind CSS

Built on Next.js 14 (App Router) with React 18. Tailwind CSS handles styling with a custom HSL-based design token system that supports dark mode out of the box. Zustand manages client-side state, and Recharts renders all data visualisations.

%¶ Backend – MongoDB & NextAuth

Next.js API Routes serve as the backend. Mongoose 8 models map to MongoDB Atlas collections for Users, Workouts, Nutrition entries, and Progress records. NextAuth handles session management and can be extended to support OAuth providers or the new role system.

%¶ REST API Endpoints for CRUD Operations

POST /api/auth/register — new user registration.

POST /api/auth/[...nextauth] — NextAuth login.

GET / POST /api/workouts — list or create workout entries.

GET / POST /api/nutrition — list or create nutrition logs.

GET / POST /api/progress — list or create progress entries.

GET / PUT /api/user — fetch or update profile and preferences.

GET /api/exercises/search — exercise suggestions via wger API.

%¶ Security & Compliance

Passwords are never stored in plain text (bcryptjs). All data-mutating API routes require a valid NextAuth session. For PT features handling health data, GDPR compliance (EU) or HIPAA alignment (US) should be evaluated before launch.

4. Development Guidelines

Follow the steps below to set up a local development environment and contribute to the project.

%¶ Clone the Repository

```
git clone https://github.com/SaidInBytes/fittrack-analytics.git
cd fittrack-analytics
```

%¶ Install Dependencies

npm install

%¶ Configure Environment Variables

Create a file named `.env.local` in the project root. Replace every placeholder (shown in angle brackets) with your actual values:

```
MONGODB_URI=mongodb+srv://<your-user>:<your-password>@<your-cluster>.mongodb.net/fittrack
NEXTAUTH_SECRET=<your-random-secret>
NEXTAUTH_URL=http://localhost:3000
WGER_API_KEY=<your-wger-api-key> # optional
```

%¶ Start the Development Server

npm run dev

Open `http://localhost:3000` in your browser. If you change `.env.local` while the server is running, restart `npm run dev` to reload the environment.

%¶ Lint Before Opening a Pull Request

npm run lint

Fix any reported issues before pushing. All changes should target a feature branch created from main.
